

Assignment #6: 链表、栈和排序

Updated 2143 GMT+8 Oct 13, 2025

2025 fall, Compiled by 窦兆文 元培学院

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E24588: 后序表达式求值

Stack, <http://cs101.openjudge.cn/practice/24588/>

思路：

后序表达式的求值可以使用栈来实现：

遇到操作数，压入栈

遇到运算符，从栈中弹出两个操作数，进行运算，将结果压入栈

最后栈中剩下的唯一元素就是表达式的值

代码：

```

def evaluate_postfix(expression):
    tokens = expression.split()
    stack = []
    operators = {'+', '-', '*', '/'}

    for token in tokens:
        if token in operators:
            b = stack.pop()
            a = stack.pop()
            if token == '+':
                result = a + b
            elif token == '-':
                result = a - b
            elif token == '*':
                result = a * b
            elif token == '/':
                result = a / b

            stack.append(result)
        else:
            stack.append(float(token))

    return stack[0]

n = int(input())
for _ in range(n):
    expression = input().strip()
    result = evaluate_postfix(expression)
    print(f"{result:.2f}")

```

代码运行截图

状态: **Accepted**

源代码

```

def evaluate_postfix(expression):
    """求后序表达式的值"""
    tokens = expression.split()
    stack = []
    operators = {'+', '-', '*', '/'}

    for token in tokens:
        if token in operators:
            b = stack.pop()
            a = stack.pop()
            if token == '+':
                result = a + b
            elif token == '-':
                result = a - b
            elif token == '*':
                result = a * b
            elif token == '/':
                result = a / b

            stack.append(result)
        else:
            stack.append(float(token))

    return stack[0]

```

基本信息

#: 50410085
 题目: 24588
 提交人: 窦兆文
 内存: 3640kB
 时间: 21ms
 语言: Python3
 提交时间: 2025-10-17 13:45:45

M234.回文链表

linked list, <https://leetcode.cn/problems/palindrome-linked-list/>

请用快慢指针实现 $O(1)$ 空间复杂度。

思路：

使用快慢指针找到链表中点

反转后半部分链表

比较前后两部分

代码：

```
class Solution:
    def isPalindrome(self, head: ListNode) -> bool:
        if not head or not head.next:
            return True
        slow = fast = head
        while fast.next and fast.next.next:
            slow = slow.next
            fast = fast.next.next
        second_half = self.reverse_list(slow.next)
        first_half = head
        result = True
        while second_half:
            if first_half.val != second_half.val:
                result = False
                break
            first_half = first_half.next
            second_half = second_half.next
        slow.next = self.reverse_list(slow.next)

        return result

    def reverse_list(self, head: ListNode) -> ListNode:
        prev = None
        curr = head
        while curr:
            next_temp = curr.next
            curr.next = prev
            prev = curr
            curr = next_temp
        return prev
```

代码运行截图

通过 93 / 93 个通过的测试用例

Youthful Spencezwr 提交于 2025.10.17 13:51

官方题解

写题解



高频 SQL 50 题

面试考点全覆盖应对高阶数据库面试



① 执行用时分布



850 ms | 击败 6.87%

复杂度分析

消耗内存分布

180.74 MB | 击败 5.02%

M27217: 有多少种合法的出栈顺序

<http://cs101.openjudge.cn/practice/27217/>

思路:

输入序列必须按 1, 2, 3, ..., n 的顺序进栈

可以在任意时刻进行 pop 操作

每个元素必须先 push 再 pop

这等价于: n 对括号的合法匹配数

代码:

```
def catalan_dp(n):
    if n == 0 or n == 1:
        return 1
    dp = [0] * (n + 1)
    dp[0] = dp[1] = 1
    for i in range(2, n + 1):
        for j in range(i):
            dp[i] += dp[j] * dp[i - 1 - j]
    return dp[n]

if __name__ == "__main__":
    n = int(input())
    result = catalan_dp(n)
    print(result)
```

代码运行截图

状态: **Accepted**

源代码

```
def catalan_dp(n):  
    if n == 0 or n == 1:  
        return 1
```

基本信息

#: 50410225

题目: 27217

提交人: 窦兆文

内存: 3696kB

时间: 241ms

M24591:中序表达式转后序表达式

<http://cs101.openjudge.cn/practice/24591/>

思路:

遇到数字: 直接加入输出队列

遇到左括号 (: 压入运算符栈

遇到右括号):

不断弹出栈顶运算符到输出队列

直到遇到左括号为止

左括号弹出但不加入输出

遇到运算符:

如果栈顶运算符优先级 $\geq$ 当前运算符, 弹出到输出队列

重复上述过程, 直到栈空或遇到左括号或栈顶优先级更低

当前运算符压入栈

表达式结束: 将栈中剩余运算符依次弹出到输出队列

代码

```

def infix_to_postfix(expression):
    precedence = {'+': 1, '-': 1, '*': 2, '/': 2}

    output = []
    stack = []
    i = 0

    while i < len(expression):
        char = expression[i]
        if char.isdigit() or char == '.':
            num = ''
            while i < len(expression) and (expression[i].isdigit() or expression[i] == '.'):
                num += expression[i]
                i += 1
            output.append(num)
            continue
        elif char == '(':
            stack.append(char)
        elif char == ')':
            while stack and stack[-1] != '(':
                output.append(stack.pop())
            if stack:
                stack.pop()
        elif char in precedence:
            while (stack and
                   stack[-1] != '(' and
                   stack[-1] in precedence and
                   precedence[stack[-1]] >= precedence[char]):
                output.append(stack.pop())
            stack.append(char)
        i += 1
    while stack:
        output.append(stack.pop())
    return ' '.join(output)

n = int(input())
for _ in range(n):
    expression = input().strip()
    result = infix_to_postfix(expression)
    print(result)

```

![alt text](6.4-1.png)

M02299:Ultra-QuickSort

merge sort, <http://cs101.openjudge.cn/practice/02299/>

思路：

代码

(至少包含有"Accepted")

M146.LRU缓存

hash table, doubly-linked list, <https://leetcode.cn/problems/lru-cache/>

思路：

交换次数 = 逆序对的个数

代码：

```

def merge_sort_and_count(arr):
    if len(arr) <= 1:
        return arr, 0
    left_arr = arr[:mid]
    right_arr = arr[mid:]
    left_sorted, left_inv = merge_sort_and_count(left_arr)
    right_sorted, right_inv = merge_sort_and_count(right_arr)
    merged, merge_inv = merge_and_count(left_sorted, right_sorted)
    total_inv = left_inv + right_inv + merge_inv
    return merged, total_inv

def merge_and_count(left, right):
    merged = []
    inversions = 0
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            inversions += len(left) - i
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged, inversions

def count_swaps(arr):
    _, inversions = merge_sort_and_count(arr)
    return inversions

while True:
    n = int(input())
    if n == 0:
        break

    arr = []
    for _ in range(n):
        arr.append(int(input()))

    result = count_swaps(arr)
    print(result)

```

代码运行截图

状态: Accepted

源代码

```
def merge_sort_and_count(arr):  
    """  
    归并排序并统计逆序对  
    """  
    if len(arr) <= 1:  
        return arr, 0  
    mid = len(arr) // 2  
    left, left_count = merge_sort_and_count(arr[:mid])  
    right, right_count = merge_sort_and_count(arr[mid:])  
    merged, merge_count = merge(left, right)  
    return merged, left_count + right_count + merge_count
```

基本信息

#: 50410314

题目: 02299

提交人: 窦兆文

语言: C++

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025fall每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

通过这几道题目，对栈和链表的应用有了进一步的认识：

栈的应用：

后序表达式求值：用栈存储操作数，遇到运算符就弹出两个数计算

中序转后序：用栈暂存运算符，按优先级控制输出顺序

出栈序列计数：理解栈操作的本质，引出卡特兰数

链表的操作：

快慢指针技巧：找中点、检测环

链表反转：经典的三指针法